Structured outputs

You can configure Gemini models to generate responses that adhere to a provided JSON Schema. This ensures predictable, type-safe results and simplifies extracting structured data from unstructured text.

Using structured outputs is ideal for:

- **Data extraction:** Pull specific information like names and dates from text.
- **Structured classification:** Classify text into predefined categories.
- **Agentic workflows:** Generate structured inputs for tools or APIs.

In addition to supporting JSON Schema in the REST API, the Google GenAI SDKs allow defining schemas using Pydantic (https://docs.pydantic.dev/latest/) (Python) and Zod (https://zod.dev/) (JavaScript).

Structured output examples

Recipe Extractor

This example demonstrates how to extract structured data from text using basic JSON Schema types like `object`, `array`, `string`, and `integer`.

Python JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)
(#python)

```
from google import genai
from pydantic import BaseModel, Field
from typing import List, Optional

class Ingredient(BaseModel):
    name: str = Field(description="Name of the ingredient.")
    quantity: str = Field(description="Quantity of the ingredient, including units.")

class Recipe(BaseModel):
    recipe_name: str = Field(description="The name of the recipe.")
    prep_time_minutes: Optional[int] = Field(description="Optional time in minutes to prepare the recipe.")
    ingredients: List[Ingredient]
    instructions: List[str]

client = genai.Client()

prompt = """
Please extract the recipe from the following text.
The user wants to make delicious chocolate chip cookies.
They need 2 and 1/4 cups of all-purpose flour, 1 teaspoon of baking soda,
1 teaspoon of salt, 1 cup of unsalted butter (softened), 3/4 cup of granulated sugar,
3/4 cup of packed brown sugar, 1 teaspoon of vanilla extract, and 2 large eggs.
For the best part, they'll need 2 cups of semisweet chocolate chips.
First, preheat the oven to 375°F (190°C). Then, in a small bowl, whisk together the flour,
baking soda, and salt. In a large bowl, cream together the butter, granulated sugar, and brown sugar
until light and fluffy. Beat in the vanilla and eggs, one at a time. Gradually beat in the dry
ingredients until just combined. Finally, stir in the chocolate chips. Drop by rounded tablespoons
onto ungreased baking sheets and bake for 9 to 11 minutes.
"""

interaction = client.interactions.create(
```

```
    model="gemini-3.8-flash",
    input=prompt,
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": Recipe.model_json_schema()
    },
)

recipe = Recipe.model_validate_json(interaction.output_text)
print(recipe)
```

Example Response:

```
{
  "recipe_name": "Delicious Chocolate Chip Cookies",
  "ingredients": [
    { "name": "all-purpose flour", "quantity": "2 and 1/4 cups" },
    { "name": "baking soda", "quantity": "1 teaspoon" },
    { "name": "salt", "quantity": "1 teaspoon" },
    { "name": "unsalted butter (softened)", "quantity": "1 cup" },
    { "name": "granulated sugar", "quantity": "3/4 cup" },
    { "name": "packed brown sugar", "quantity": "3/4 cup" },
    { "name": "vanilla extract", "quantity": "1 teaspoon" },
    { "name": "large eggs", "quantity": "2" },
    { "name": "semisweet chocolate chips", "quantity": "2 cups" }
  ],
  "instructions": [
    "Preheat the oven to 375°F (190°C).",
    "In a small bowl, whisk together the flour, baking soda, and salt.",
```

```
    "In a large bowl, cream together the butter, granulated sugar, and brown sugar until light and fluffy.",
    "Beat in the vanilla and eggs, one at a time.",
    "Gradually beat in the dry ingredients until just combined.",
    "Stir in the chocolate chips.",
    "Drop by rounded tablespoons onto ungreased baking sheets and bake for 9 to 11 minutes."
  ]
}
```

Content Moderation

This example showcases `anyOf` for conditional schemas and `enum` for classification, allowing the output structure to vary based on the content.

Python JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)
(#python)

```
from google import genai
from pydantic import BaseModel, Field
from typing import Union, Literal

class SpamDetails(BaseModel):
    reason: str = Field(description="The reason why the content is considered spam.")
    spam_type: Literal["phishing", "scam", "unsolicited promotion", "other"] = Field(description="The type of sp

class NotSpamDetails(BaseModel):
    summary: str = Field(description="A brief summary of the content.")
    is_safe: bool = Field(description="Whether the content is safe for all audiences.")

class ModerationResult(BaseModel):
```

```
    decision: Union[SpamDetails, NotSpamDetails]

client = genai.Client()

prompt = """
Please moderate the following content and provide a decision.
Content: 'Congratulations! You''ve won a free cruise to the Bahamas. Click here to claim your prize: www.defini
"""

interaction = client.interactions.create(
    model="gemini-3.8-flash",
    input=prompt,
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": ModerationResult.model_json_schema()
    },
)

result = ModerationResult.model_validate_json(interaction.output_text)
print(result)
```

Example Response:

```
{
  "decision": {
    "reason": "The content is an unsolicited prize notification attempting to trick the user into clicking a suspicio
    "spam_type": "scam"
```

```
  }
}
```

Recursive Structures

This example illustrates how to define a recursive schema such as an organization chart.

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)

```
from google import genai
from pydantic import BaseModel, Field
from typing import List

class Employee(BaseModel):
    """Represents an employee in an organization."""
    name: str
    employee_id: int
    reports: List["Employee"] = Field(
        default_factory=list,
        description="A list of employees reporting to this employee."
    )

client = genai.Client()

prompt = """
Generate an organization chart for a small team.
The manager is Alice, who manages Bob and Charlie. Bob manages David.
"""
```

```
interaction = client.interactions.create(
    model="gemini-3.8-flash",
    input=prompt,
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": Employee.model_json_schema()
    },
)

employee = Employee.model_validate_json(interaction.output_text)
print(employee)
```

Example Response:

```
{
  "name": "Alice",
  "employee_id": 101,
  "reports": [
    {
      "name": "Bob",
      "employee_id": 102,
      "reports": [
        {
          "name": "David",
          "employee_id": 104,
          "reports": []
        }
      ]
```

```
    },
    {
      "name": "Charlie",
      "employee_id": 103,
      "reports": []
    }
  ]
}
```

Streaming results

You can stream structured outputs, allowing you to start processing the response as it's being generated. The streamed chunks are valid partial JSON strings that can be concatenated to form the final JSON object.

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)

```
from google import genai
from pydantic import BaseModel
from typing import Literal

class Feedback(BaseModel):
    sentiment: Literal["positive", "neutral", "negative"]
    summary: str

client = genai.Client()
prompt = "The new UI is incredibly intuitive. Add a very long summary to test streaming!"
```

```
stream = client.interactions.create(
    model="gemini-3.8-flash",
    input=prompt,
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": Feedback.model_json_schema()
    },
    stream=True
)
for event in stream:
    if event.event_type == "step.delta":
        if event.delta.type == "text" and getattr(event.delta, "text", None):
            print(event.delta.text, end="", flush=True)
```

Structured outputs with tools

Preview: This feature is available only to Gemini 3 series models.

Gemini 3 lets you combine Structured Outputs with built-in tools, including Grounding with Google Search (/gemini-api/docs/google-search), URL Context (/gemini-api/docs/url-context), Code Execution (/gemini-api/docs/code-execution), File Search (/gemini-api/docs/file-search#structured-output), and Function Calling (/gemini-api/docs/function-calling).

Python (#python) JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)

```
from google import genai
from pydantic import BaseModel, Field
from typing import List

class MatchResult(BaseModel):
    winner: str = Field(description="The name of the winner.")
    final_match_score: str = Field(description="The final match score.")
    scorers: List[str] = Field(description="The name of the scorer.")

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-4-argon",
    input="Search for all details for the latest Euro.",
    tools=[{"type": "google_search"}, {"type": "url_context"}],
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": MatchResult.model_json_schema()
    },
)

result = MatchResult.model_validate_json(interaction.output_text)
print(result)
```

JSON schema support

To generate a JSON object, configure `response_format` with an object (or an array containing an object) of type `text` and set its `mime_type` to `application/json`. The schema should be provided in the `schema` field.

Gemini's structured output mode supports a subset of the JSON Schema (https://json-schema.org/) specification.

The following values of `type` are supported:

- `string`: For text.
- `number`: For floating-point numbers.
- `integer`: For whole numbers.
- `boolean`: For true or false values.
- `object`: For structured data with key-value pairs.
- `array`: For lists of items.
- `null`: To allow a property to be null, include `"null"` in the type array (e.g., `{"type": ["string", "null"]}`).

These descriptive properties help guide the model:

- `title`: A short description of a property.
- `description`: A longer and more detailed description of a property.

Type-specific properties

For `object` values:

- `properties`: An object where each key is a property name and each value is a schema for that property.

- `required`: An array of strings, listing which properties are mandatory.
- `additionalProperties`: Controls whether properties not listed in `properties` are allowed. Can be a boolean or a schema.

For `string` values:

- `enum`: Lists a specific set of possible strings for classification tasks.
- `format`: Specifies a syntax for the string, such as `date-time`, `date`, `time`.

For `number` and `integer` values:

- `enum`: Lists a specific set of possible numeric values.
- `minimum`: The minimum inclusive value.
- `maximum`: The maximum inclusive value.

For `array` values:

- `items`: Defines the schema for all items in the array.
- `prefixItems`: Defines a list of schemas for the first N items, allowing for tuple-like structures.
- `minItems`: The minimum number of items in the array.
- `maxItems`: The maximum number of items in the array.

Structured outputs versus function calling

Feature	Primary Use Case
Structured Outputs	**Formatting the final response.** Use when you want the model's *answer* in a specific format.
Function Calling	**Taking action during conversation.** Use when the model needs to *ask you* to perform a task before providing a final answer.

Best practices

- **Clear descriptions:** Use the `description` field to guide the model.
- **Strong typing:** Use specific types (`integer`, `string`, `enum`).
- **Prompt engineering:** Clearly state what you want the model to do.
- **Validation:** While output is syntactically correct JSON, always validate values in your application.
- **Error handling:** Implement robust error handling for schema-compliant but semantically incorrect outputs.

Limitations

- **Schema subset:** Not all JSON Schema features are supported.
- **Schema complexity:** Very large or deeply nested schemas may be rejected.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-10-13 UTC.